

BBVOLTEX — Rapport de projet complet

Document de suivi généré le 20 septembre 2026, à l'état du commit `b5a26bf` sur la branche `claude/ecommerce-kids-toys-clothing-2anm9e`. Dépôt : [sosodegral-design/BBhappy](https://github.com/sosodegral-design/BBhappy) Site en ligne : <https://bbhappy.onrender.com>

1. Résumé du projet

1.1 Objectif

BBVOLTEX est un site e-commerce (jouets, vêtements enfants, bijoux & accessoires, box surprises, déstockage...) avec un vrai backend — pas une maquette statique. Le panier, les favoris, les points de fidélité et les commandes sont gérés côté serveur et persistent réellement (base de données), avec un vrai paiement Stripe en production.

1.2 État d'avancement actuel

- **Site fonctionnel et déployé** sur Render, connecté à une base **Turso** (SQLite distant, persistant entre les redéploiements).
- **20 pages HTML** couvrant toutes les catégories, la fidélité, le suivi de commande, le FAQ, les pages légales/informatives, etc.
- **58 produits** au catalogue, répartis sur 9 catégories (voir §2.4).
- Catégorie **"Bijoux & Accessoires"** développée en profondeur cette session : 3 colliers avec vraies photos fournisseur, descriptions complètes bilingues FR/EN, couleurs, galerie multi-photos.
- **Panier, favoris, fidélité, checkout Stripe (+ mode local sans Stripe), suivi de commande** : tous fonctionnels de bout en bout, testés manuellement (Playwright + tests utilisateur réels).
- Site **bilingue FR/EN** (toggle dans le header, `i18n` maison dans `js/script.js`).

1.3 Fonctionnalités clés mises en place

Fonctionnalité	Détail
Catalogue produit	Servi par l'API (<code>GET /api/products</code>), jamais codé en dur côté navigateur
Panier persistant	Cookie visiteur anonyme + SQLite/Turso, survit aux rechargements
Favoris	Idem, par visiteur
Points de fidélité	Calculés au checkout, paliers de récompense (<code>LOYALTY_TIERS</code>)
Checkout	Stripe Checkout hébergé si <code>STRIPE_SECRET_KEY</code> est configurée, sinon commande créée directement (mode dev sans compte Stripe)
Confirmation de commande	Récapitule chaque article avec sa couleur exacte (nom lisible, pas de code hexadécimal)
Suivi de commande	<code>GET /api/track</code> , statut simulé de façon déterministe selon l'ancienneté de la commande
Recherche	Recherche globale multi-catégories, résultats groupés par catégorie, auto-scroll
Filtres catégorie	Chaque page catégorie a des tuiles "univers" (ex. Collier/Bracelet/Bague...) qui groupent automatiquement les produits sous un titre quand aucun filtre n'est actif
Couleurs produit	Chaque couleur peut avoir sa propre galerie photo (<code>imagesByColor</code>) — cliquer une couleur change réellement la photo affichée (carte, modale, panier, zoom)
Zoom photo	Lightbox plein écran avec navigation précédent/suivant
Auto-guérison image	Une photo qui échoue à charger est retentée automatiquement (jusqu'à 4 fois) avant de basculer sur l'icône de secours

2. Architecture et code

2.1 Structure des dossiers

```
BBhappy/
├── server/
│   ├── index.js          # Serveur Express + toutes les routes /api/*
│   ├── db.js             # Connexion SQLite/Turso + schéma (CREATE TABLE)
│   ├── products-data.js  # Catalogue produit (source de vérité), ICONS, LOYALTY_TIERS
│   └── bbhappy.db        # Base SQLite locale (dev only, ignorée par git)
├── js/
│   └── script.js         # Tout le JS front (il8n, panier, modale produit, checkout...)
├── css/
│   └── style.css         # Tout le CSS du site
├── images/
│   ├── favicon.png
│   └── products/        # Photos produit (dont les 12 photos des 3 bijoux)
├── scripts/
│   └── alibaba-import/  # Outil local (hors serveur) pour scraper Alibaba depuis le PC de l'utilisateur
├── *.html               # 20 pages (voir §2.3)
├── package.json
├── render.yaml          # Config de déploiement Render (build/start command, env vars)
├── .node-version        # 22.22.2
└── sitemap.xml          # Soumis à Google Search Console
```

2.2 Stack technique

- **Backend** : Node.js 22, Express 4
- **Base de données** : `@libsql/client` — SQLite local en dev (`server/bbhappy.db`), **Turso** (SQLite hébergé) en production via `TURSO_DATABASE_URL` / `TURSO_AUTH_TOKEN`
- **Paiement** : Stripe Checkout (`stripe` npm package), optionnel — le site fonctionne sans (mode "commande directe" pour le dev local)
- **Frontend** : HTML/CSS/JS vanilla, **aucun framework** (pas de React/Vue), un seul fichier `js/script.js` partagé par toutes les pages
- **Hébergement** : Render.com (plan free), déploiement automatique sur push vers la branche

2.3 Les 20 pages HTML

Catégories : `jouets`, `vetements`, `electronique`, `maison`, `beaute`, `sport`, `bijoux-accessoires`, `box`, `destockage`, `soldes`.
Autres : `index` (accueil), `carte-fidelite`, `suivi` (tracking commande), `faq`, `guide-tailles`, `livraison-retours`, `notre-histoire`, `engagement-eco`, `normes-securite`, `carrieres`.

⚠ Point d'architecture important : la modale panier/checkout (le bloc `#checkoutModal` avec `#checkoutOrderItems` etc.) est **dupliquée** dans **chacune des 20 pages HTML**, il n'y a pas de composant/template partagé. Toute modification de cette modale doit être répercutée sur les 20 fichiers (un script `perl -0pi` en bulk a été utilisé plusieurs fois cette session pour ça — voir §2.6).

2.4 Catalogue produit (`server/products-data.js`)

```
module.exports = { ICONS, PRODUCTS, LOYALTY_TIERS, colorLabel };
```

- **58 produits** au total, répartition : `vetements` (19), `jouets` (13), `box` (6), `destockage` (4), `maison` (4), `bijoux` (3), `electronique` (3), `beaute` (3), `sport` (3).
- La plupart des produits utilisent une **icône SVG dessinée** (`ICONS` , génériques, sans photo réelle).
- **Exception** : **les 3 bijoux** (`bj1`, `bj2`, `bj3`) ont de vraies photos fournisseur (Alibaba), retravaillées et intégrées cette session.

Forme d'un produit type :

```
{
  id: 'bj1', category: 'bijoux', universe: 'collier', age: 'all',
  price: 19.99, oldPrice: null, icon: ICONS.collier,
```

```

images: ['images/products/bj1-1.jpg', /* ... */],
imagesByColor: { // optionnel : galerie par couleur
  '#d4af37': ['images/products/bj1-1.jpg', /* ... */],
  '#e8b4a8': ['images/products/bj2-6.jpg']
},
bg: '#f2ead6',
colors: ['#d4af37', '#e8b4a8'],
sale: false,
name: 'Collier personnalisé pierre de naissance & pendentif cœur',
name_en: 'Custom Birthstone & Heart Pendant Necklace',
ageLabel: 'Bijoux', ageLabel_en: 'Jewelry',
description: '...', description_en: '...', // multi-paragaphes (\n\n)
ecoDetails: '...', safety: '...', care: '...',
sizeGuide: [['Caractéristique', 'Détail'], ...], // tableau de specs
sizeGuide_en: [...]
}

```

2.5 Backend — routes API (`server/index.js` , 473 lignes)

```

GET    /api/products           → catalogue complet
GET    /api/cart               → panier du visiteur courant
POST   /api/cart/items        → { productId, color, qty }
DELETE /api/cart/items/:itemId
DELETE /api/cart          → vide le panier
GET    /api/favorites
POST   /api/favorites/:productId
DELETE /api/favorites/:productId
GET    /api/loyalty
POST   /api/loyalty/bonus
POST   /api/loyalty/physical-card
POST   /api/checkout      → crée une session Stripe (ou la commande directe si pas de Stripe configuré)
GET    /api/checkout/confirm → confirme le paiement Stripe, crée la commande, renvoie orderNumber + items[]
GET    /api/track          → statut de livraison simulé (déterministe selon l'ancienneté)

```

Identification visiteur : cookie anonyme `bbhappy_cid` posé au premier chargement (pas de compte utilisateur), utilisé comme clé pour panier, favoris et fidélité.

2.6 Schéma base de données (`server/db.js`)

```

CREATE TABLE cart_items (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  cart_id TEXT NOT NULL, product_id TEXT NOT NULL,
  color TEXT NOT NULL, qty INTEGER NOT NULL,
  UNIQUE(cart_id, product_id, color)
);
CREATE TABLE favorites (cart_id TEXT NOT NULL, product_id TEXT NOT NULL, PRIMARY KEY (cart_id, product_id));
CREATE TABLE loyalty (cart_id TEXT PRIMARY KEY, points INTEGER NOT NULL DEFAULT 0);
CREATE TABLE orders (
  id TEXT PRIMARY KEY, cart_id TEXT NOT NULL,
  name TEXT, email TEXT, address TEXT, zip TEXT, city TEXT,
  total REAL NOT NULL, points_earned INTEGER NOT NULL,
  created_at TEXT NOT NULL, stripe_session_id TEXT UNIQUE
);
CREATE TABLE order_items (order_id TEXT NOT NULL, product_id TEXT NOT NULL, color TEXT NOT NULL, qty INTEGER NOT NULL, price REAL NOT NULL);
CREATE TABLE physical_card_requests (cart_id TEXT PRIMARY KEY, name TEXT NOT NULL, address TEXT NOT NULL, zip TEXT NOT NULL, city TEXT NOT NULL);

```

2.7 Frontend — points clés (`js/script.js` , 1766 lignes)

- `state` : objet global (filtre catégorie/âge/univers, langue, panier, favoris, produit ouvert dans la modale, couleur/galerie sélectionnée...).
- `TRANSLATIONS` : dictionnaire i18n FR/EN, fonction `t(key)`.
- `getFilteredProducts()` / `renderProducts()` : logique de filtre + **groupement automatique par univers** (ex. tous les colliers sous un titre "Collier") quand aucun filtre explicite n'est actif.
- `productImagesForColor(p, color)` : renvoie `imagesByColor[color]` si défini, sinon la galerie par défaut — utilisé partout où une couleur peut changer la photo affichée (carte, modale, panier, lightbox).

- `openProductModal(id)` : construit toute la fiche produit (photos, description multi-paragraphe, tableau de tailles/specs, onglet dynamique "Emballage & Fabricant" pour les bijoux).
- `wireImageRetry(container, p)` : filet de sécurité — une image qui ne charge pas est retentée avec un `?retry=` cache-buster avant de basculer sur l'icône SVG.
- **Panier / checkout** : `addToCart`, `renderCart`, `renderCheckoutSummary`, `showOrderSuccess` — **chaque ligne affiche le nom de la couleur en toutes lettres** (`colorName()`), jamais juste un code hexadécimal.

2.8 Convention de cache-busting

Les balises `<script src="js/script.js?v=N">` et `<link href="css/style.css?v=N">` portent un numéro de version manuel, présent identiquement sur les 20 pages. **Ce numéro doit être incrémenté à chaque modification de `script.js` ou `style.css`**, sinon les navigateurs des visiteurs peuvent continuer à servir une version en cache. Version actuelle : **v=7**.

3. Commandes et procédures

3.1 Installation et lancement en local

```
npm install
npm start          # démarre sur http://localhost:3000, SQLite local (server/bbhappy.db)
# ou, pour un rechargement auto du serveur pendant le dev :
npm run dev
```

Aucune configuration n'est nécessaire pour développer en local : sans `TURSO_DATABASE_URL` / `TURSO_AUTH_TOKEN`, le serveur utilise un fichier SQLite local ; sans `STRIPE_SECRET_KEY`, le checkout crée la commande directement (sans paiement réel), ce qui permet de tester tout le tunnel d'achat sans compte Stripe.

3.2 Variables d'environnement (production, sur Render)

Variable	Rôle
<code>NODE_VERSION</code>	<code>22.22.2</code> (fixée dans <code>render.yaml</code>)
<code>TURSO_DATABASE_URL</code>	URL de la base Turso (persistance réelle)
<code>TURSO_AUTH_TOKEN</code>	Jeton d'auth Turso
<code>STRIPE_SECRET_KEY</code>	Clé secrète Stripe (<code>sk_test...</code> ou <code>sk_live...</code>) — active le vrai paiement

3.3 Déploiement

Le déploiement est **automatique** sur Render à chaque `git push` sur la branche `claude/ecommerce-kids-toys-clothing-2anm9e` :

```
git add -A
git commit -m "... "
git push -u origin claude/ecommerce-kids-toys-clothing-2anm9e
```

Render détecte le push, exécute `npm install` puis `npm start` (voir `render.yaml`), et le site est live sur **`https://bbhappy.onrender.com`** en général en moins d'une minute.

3.4 Vérifier le code avant de pousser

```
node --check server/index.js
node --check server/products-data.js
node --check js/script.js
```

3.5 Tester visuellement en local (utilisé cette session)

```
node server/index.js & # lance le serveur local
# Playwright (Chromium pré-installé dans cet environnement) pour capturer
# des captures d'écran et valider un parcours (ajout panier, checkout...)
```

3.6 Carte de test pour valider un paiement Stripe (mode test uniquement)

```
Numéro : 4242 4242 4242 4242
Expiration : n'importe quelle date future
CVC : n'importe quel 3 chiffres
Code postal : n'importe lequel
```

⚠ Ne fonctionne que si `STRIPE_SECRET_KEY` est une clé **test** (`sk_test_...`). Avec une clé live, elle est simplement refusée (aucun risque de débit réel).

4. Tâches restantes et pistes d'amélioration

4.1 Catalogue à compléter

- **~49 produits** de la catégorie Bijoux & Accessoires restent à ajouter (bracelets, bagues, ceintures, montres, sacs à main, sacoches/sacs à dos — seul "Collier" a des produits pour l'instant sur cette catégorie).
- Workflow établi et qui fonctionne : l'utilisateur sauvegarde manuellement de vraies photos depuis Alibaba (clic droit → enregistrer, le scraping automatique est bloqué par un captcha anti-bot), les envoie via l'upload GitHub, colle les infos/specs du produit dans la conversation ; la fiche produit (description bilingue, tableau de specs, couleurs) est ensuite construite à partir de ça.
- La plupart des autres catégories (jouets, vêtements...) utilisent encore des **icônes SVG génériques**, pas de vraies photos.

4.2 Photos produit — statut actuel

- Une tentative d'uniformisation des photos des 3 colliers (fond crème uni + détourage + retouche "éclat") a été faite puis **entièrement annulée** à la demande de l'utilisateur ("elles sont moches") — commit `b5a26bf`. Les photos originales fournisseur (chacune avec son propre décor) sont actuellement en ligne.
- Si une nouvelle direction artistique est souhaitée pour les photos, elle reste à définir avec l'utilisateur avant toute nouvelle tentative automatisée (fond, cadrage, niveau de retouche).

4.3 Bugs connus / points de vigilance

- **Duplication de la modale checkout sur 20 fichiers HTML** (§2.3) : source d'erreurs si une future modification n'est appliquée qu'à `index.html` (déjà arrivé une fois cette session — corrigé après coup).
- **Convention `?v=N`** de cache-busting facile à oublier après une modification JS/CSS ; un oubli peut faire croire à un bug côté utilisateur alors qu'il s'agit juste de cache navigateur.
- Le catalogue mélange des produits avec vraies photos et produits avec icônes ; pas de fallback visuel "photo manquante" autre que l'icône SVG générique.

4.4 Améliorations possibles (non demandées explicitement, à discuter)

- Extraire la modale checkout dans un template/partial unique (via un script de build ou des includes) pour éviter la duplication sur 20 fichiers.
- Ajouter les articles de la commande (avec couleur) à l'écran de **suivi de commande** (`/api/track` ne renvoie actuellement que le statut, pas le détail des articles).
- Étendre la recherche/le filtrage univers à toutes les catégories qui n'en ont pas encore.
- Réfléchir à une direction photo cohérente (avec l'utilisateur) pour les futurs produits bijoux, y compris la question d'une éventuelle photo "portée" par un modèle — nécessiterait soit de vraies photos fournisseur supplémentaires, soit un outil de génération d'image (non disponible dans cet environnement actuellement).

5. Historique de session (résumé chronologique)

1. Mise en place d'un outil de scraping Alibaba **local** (pour le PC de l'utilisateur, jamais exécuté côté serveur — bloqué par anti-bot).
2. Création de la page/catégorie **Bijoux & Accessoires** (7 univers, icônes, traductions).
3. Ajout du premier collier réel (**bj1**) avec vraies photos, description détaillée bilingue, tableau de specs.
4. Diagnostic et correction d'un bug Firefox subtil (`loading="lazy"` sur une image affichée via toggle CSS → requête jamais déclenchée) + mécanisme d'auto-retry en filet de sécurité.
5. Ajout de 2 colliers supplémentaires (**bj2** double cœur, **bj3** lune & étoile), conversion de photos mal étiquetées (AVIF/PNG déguisées en `.jpg`) en vrais JPEG compressés.
6. Regroupement automatique des produits par "univers" sur les pages catégorie (au lieu d'une liste en vrac).
7. Système de **galerie par couleur** (`imagesByColor`) : cliquer une couleur change réellement la photo (carte, modale, panier, zoom).
8. Correction d'un vrai bug de confiance client : la couleur choisie n'apparaissait nulle part dans la confirmation de commande (ni sur la page Stripe elle-même, qui montrait un code hex brut) — corrigé à tous les niveaux (toast, panier, récapitulatif, confirmation finale).
9. Tentative d'uniformisation esthétique des photos (suppression de fond + retouche), **annulée** sur retour utilisateur négatif → retour aux photos originales.

Fin du rapport. Pour reprendre le travail, ce fichier + le dépôt Git (historique de commits complet) contiennent tout le contexte nécessaire.